

Warehouse Image Classification Pipeline

Overview

Binary satellite image classifier: **warehouse vs. non-warehouse**. Input: Google Maps Static API tiles (640×640, zoom=18, satellite). No bounding boxes or footprints — classification only. Images are downloaded at a fixed resolution (640×640, zoom=18) via the Google Maps Static API, requiring no additional preprocessing. Normalization and resizing are handled internally by the YOLO pipeline.

Pipeline runs across three notebooks in order:

1. Sampling Pipeline → builds the negative image dataset
 2. YOLO Train & Result → first training run
 3. YOLO Comparison → second training run with adjusted settings
-

Notebook 1 — Sampling Pipeline

1.1 Positive Data

Three raw KML files from Loadsmart warehouse records are cleaned, merged, and deduplicated on (lat, lon). **Final: 4,205 unique warehouse points.**

1.2 Negative Data Strategy — Rationale

The assignment only provides positive examples. While external building datasets exist (e.g. Google Open Buildings, Microsoft Building Footprints), these provide building footprint coordinates rather than image-level labels — making them better suited for detection or segmentation tasks. Adapting them for a classification pipeline would require fetching satellite tiles per coordinate, aligning geometries, and manually assigning warehouse/non-warehouse labels, with no clear gain over building negatives from scratch.

OSM was chosen instead for three reasons:

- **Task alignment:** OSM provides named place categories that map directly to image-level labels, which is exactly what a classifier needs.
- **Domain control:** categories can be selected to be geographically realistic (places a warehouse classifier would actually encounter in production), avoiding distribution mismatch.
- **Visual consistency:** negatives are fetched from the same source (Google Maps Static API, same zoom and tile size) as the positives, keeping the image domain uniform across classes.

1.3 Density Labeling

DBSCAN is applied to positive points to capture local warehouse density patterns. This informs how aggressively to search for negatives in different areas.

- Parameter sweep: $\text{eps} = [7.5, 10, 15] \text{ km} \times \text{min_samples} = [5, 8, 12]$
- **Final setting: $\text{eps} = 10 \text{ km}$, $\text{min_samples} = 5$**

- Each positive is labeled: `isolated` / `small_cluster` / `dense_cluster`

1.4 Negative Sampling Strategies

Three strategies were designed with varying degrees of density-awareness.

A fourth (density_v3) was discarded because its exclusion buffer exceeded the search radius,

producing near-zero candidates for dense-cluster seeds.

Strategy	Approach	isolated	small_cluster	dense_cluster
<code>baseline_global</code>	flat params (control)	2500m / 1000m	2500m / 1000m	2500m / 1000m
<code>density_v1</code>	moderate differentiation	4000m / 900m	2500m / 1100m	1500m / 1500m
<code>density_v2</code>	aggressive differentiation	5000m / 800m	3000m / 1200m	1800m / 1600m

(format: search_dist / exclusion_buffer)

density_v1 showed limited difference from baseline in downstream evaluation. Final comparison focuses on baseline_global vs. density_v2 only.

1.5 Standard OSM Negative Sampling

10 categories queried from OpenStreetMap, targeting 100 images each (1,000 total per strategy).

Categories: `fuel`, `parking`, `restaurant`, `hospital`, `hotel`, `supermarket`, `school`, `park`, `playground`, `sports_centre`

Seed allocation per category:

- isolated: 20 seeds, small_cluster: 15 seeds, dense_cluster: 15 seeds (50 total)

Workflow per (strategy, category):

1. Sample seed points from positives by density group
2. Query OSM within `search_dist` of each seed
3. Filter out candidates within `exclusion_buffer` of any positive
4. Deduplicate across seeds, downsample to 100

1.6 Top-Up Sampling

Several categories did not reach 100 images after the initial run.

Only underfilled (strategy, category) pairs are re-queried, with relaxed search params and expanded tag sets.

Expanded tags for low-yield categories:

Category	Added tags
hospital	clinic, doctors
hotel	motel, guest_house
supermarket	wholesale
sports_centre	stadium, fitness_centre

Strategy	search_dist	exclusion_buffer
density_v2	6000m	800m
baseline_global	4000m	1000m

Output: `neg_density_v2_filled.csv`, `neg_baseline_global_filled.csv`

1.7 Hard Negative Sampling

Motivation: the earlier classifier showed ~79% accuracy regardless of region density, suggesting the main error source was visual confusion rather than spatial distribution. Warehouses are frequently misclassified against large flat-roof industrial/commercial structures.

Hard negative categories (visually similar to warehouses from satellite view):

`industrial`, `factory`, `retail_large`, `commercial`, `wholesale`

- wholesale also queries `shop=doityourself` (large DIY/hardware stores)

Key design decision: the same hard negatives are appended to **both** strategy CSVs. This keeps the baseline_global vs. density_v2 comparison controlled — hard negative quality is not a variable between the two strategies.

Param	Value
search_dist	4000m
exclusion_buffer	1000m
n_seed	40 (density-agnostic)
n_target	100 / category

Hard negatives are appended to `_filled.csv` for both strategies before training.

Notebook 2 — YOLO Train & Result (Run 1)

2.1 Dataset Construction

Split: 70% train / 15% val / 15% test (stratified by class)

Image download: Google Maps Static API, one tile per (lat, lon) point. Images saved as `{split}/{class}/{img}.png`.

Augmentation: negative training images only. Reason: positives $\approx 4,205$; negatives are fewer even after top-up and hard negatives. Augmenting only negatives reduces class imbalance.

- 2 augmented copies per original negative training image
- Combos: (horizontal flip + brightness) and (90° rotation + contrast)
- YOLO built-in augmentation: disabled

Note: YOLO sorts class folders alphabetically at runtime $\rightarrow$ `non_warehouse=0`, `warehouse=1`

2.2 Model & Training

- Model: YOLOv8s-cls
- Epochs: 50, batch: 32, imgsz: 640, patience: 10
- dropout: 0.3, lr0: 1e-3, weight_decay: 1e-4

Two models trained in parallel: one for `density_v2`, one for `baseline_global`.

2.3 Results

Classification Report (test set, 70/15/15 split):

Model	Accuracy	Warehouse F1	Non-warehouse F1	Macro F1
density_v2	0.92	0.95	0.85	0.90
baseline_global	0.94	0.96	0.88	0.92

baseline_global outperformed density_v2 in Run 1 across all metrics.

Confusion matrix highlights:

Model	TP (warehouse)	FN	FP	TN (non-wh)
density_v2	597	34	29	177
baseline_global	609	22	27	179

Non-warehouse consistently scores lower F1 than warehouse across both models, reflecting the harder classification boundary for visually ambiguous negatives.

Training curves:

Both models show clear overfitting — train loss converges to ~ 0.03 – 0.05 while val loss plateaus around 0.25 – 0.35 with high spike variance throughout training. This motivates the stronger regularization applied in Run 2.

- density_v2: early stopping at ~ 35 epochs
- baseline_global: ran full 50 epochs

Notebook 3 — YOLO Comparison (Run 2)

3.1 What Changed from Run 1

Run 2 is a retraining experiment with three adjustments aimed at stronger regularization:

Setting	Run 1	Run 2
Train/val/test split	70/15/15	60/20/20
Augmentation scope	neg only	pos + neg
dropout	0.3	0.5
weight_decay	1e-4	1e-3
YOLO built-in augment	False	True

Dataset is re-split from `manifest.csv` (successfully downloaded originals only).

3.2 Training

Same architecture (YOLOv8s-cls) and same two strategies.
Both models retrained from scratch with Run 2 settings.

3.3 Results & Comparison

Classification Report (test set, 60/20/20 split):

Model	Accuracy	Warehouse F1	Non-warehouse F1	Macro F1
density_v2	0.93	0.96	0.86	0.91
baseline_global	0.91	0.94	0.82	0.88

In Run 2, density_v2 outperforms baseline_global — the opposite of Run 1.

Confusion matrix highlights:

Model	TP (warehouse)	FN	FP	TN (non-wh)
density_v2	811	30	45	229
baseline_global	777	64	41	233

Cross-run comparison (density_v2 vs baseline_global):

	Run 1 baseline	Run 1 density_v2	Run 2 baseline	Run 2 density_v2
Accuracy	0.94	0.92	0.91	0.93
Warehouse F1	0.96	0.95	0.94	0.96
Non-wh F1	0.88	0.85	0.82	0.86
Macro F1	0.92	0.90	0.88	0.91

Key observations:

- **Overfitting reduced in Run 2:** stronger regularization (dropout 0.5, weight_decay 1e-3, augment=True) narrowed the train-val loss gap and reduced val loss spike variance, though a gap remains.
- **density_v2 benefits more from regularization:** in Run 1 it underperformed baseline; in Run 2 it reversed and outperformed. Suggests density-aware negatives produce a harder, noisier training signal that needs stronger regularization to generalise.
- **Non-warehouse remains the harder class** in all four models, consistent with the visual confusion motivation behind hard negatives.
- **Early stopping:** Run 2 density_v2 stopped at ~35 epochs; baseline_global stopped at ~20 epochs, suggesting faster convergence with the 60/20/20 split providing more val signal.

Appendix — Key Parameters Reference

Sampling strategy params (search_dist m / exclusion_buffer m)

Strategy	isolated	small_cluster	dense_cluster
baseline_global	2500 / 1000	2500 / 1000	2500 / 1000
density_v2	5000 / 800	3000 / 1200	1800 / 1600

DBSCAN final setting

eps = 10 km, min_samples = 5

Hard negative OSM tags

- industrial: `landuse=industrial`
- factory: `building=factory`
- retail_large: `building=retail`
- commercial: `building=commercial`
- wholesale: `shop=wholesale`, `shop=doityourself`

Output files

File	Contents
<code>positive_points_cleaned_dedup.csv</code>	4,205 warehouse points
<code>neg_{strategy}_filled.csv</code>	soft neg + hard neg combined
<code>manifest.csv</code>	per-image split/download record
<code>aug_log_neg.csv</code>	augmentation log

File	Contents
dbscan_parameter_sweep_summary.csv	9-combo sweep results